

Fulton Hogan Roadway Construction Analysis

Technical Framework

Authors: Brayden Davies (74597743, bda79), David Ewing (82171165, dew59), Ann Benji (67936363, abe167), Matthew Madriaga (11720432, jma538)

Date: 11th October 2024

Executive Summary

The report presents an analysis conducted for Fulton Hogan to determine the most suitable time for scheduling roadworks in the Central Business District (CBD) of Auckland, Wellington and Christchurch, using telecommunications data from spark and Vodafone. The goal of this project is to optimize roadwork scheduling by minimizing traffic disruption, reducing costs, and ensuring safety for both workers and road users.

Data from multiple sources, including Telecommunications data from spark and Vodafone, Statistical Area and Territorial Authority Data, Populations estimates and urban rural indicators was utilised. The data pre processing phase involved cleaning and preparing these data sets to ensure accuracy and consistency. Challenges such as missing or duplicate data were resolved through validation. Functions were developed to standardise data, flag, and remove errors and merging data sets. We generated maps for each CBD to identify the relevant regions, after which the analytical process could begin.

Introduction

Fulton Hogan undertakes significant roadworks across major cities in New Zealand, including Auckland, Wellington and Christchurch. These roadworks can be highly disruptive due to a variety of factors, such as costs, traffic management, and safety risks. This investigation focuses on finding the most appropriate time to plan roadworks in these cities' CBDs.

Roads in these busy urban centres often deteriorate over time or require upgrades to accomodate increasing traffic or new development projects. Regular maintenance is essential to ensure safe driving conditions, improve traffic flow, and adapt to new infrastructure requirements. The objective of this investigation is to identify optimal roadwork schedules that minimises costs, improve worker safety, and reduce traffic disruption.

We achieve this by leveraging telecommunications data from Spark and Vodafone, along with Statistical (SA2) and Territorial (TA) Area data, to determine the number of mobile phones present within specific SA2 regions.

Data Description

The data sets used in the analysis include:

- **sp_data.csv.gz** - Telecommunications data from Spark
- **vf_data.parquet** - Telecommunications data from Vodafone
- **sa2_2023.csv** - This file contains the Statistical Area Level 2 (SA2) codes with their corresponding regions recorded in 2023.
- **sa2_ta_concord_2023.csv** - This concordance report contains the SA2 codes with their corresponding regions. Additionally, it has the Territorial Authority (TA) code corresponding to its territory name. We also get the relationship of the SA2 codes to the TA codes on the map.
- **subnational_pop_ests.csv** - The subnational population estimates contain population information like gender, age and population information for the corresponding SA2 regions.

- **urban_rural_to_indicator_2023.csv** - This CSV file gives us an insight into the various types of settlement across the statistical regions of New Zealand.
 - **urban_rural_to_sa2_concord_2023.csv** - This concordance report gives the relationship between the urban rural code and the SA2 code for all the regions in New Zealand.
-

Data Preprocessing

This section outlines the data preparation process, which included cleaning and validating the datasets. Challenges such as missing values, deuplicate records, and incorrect column names were addressed using a series of scripts developed for standardising the data.

Refer to Ewing, D. (2024). Narrative and debugging overview of cellphone data cleaning functions for a detailed description of the data cleaning process.

(20241010_Narrative_and_Debugging_Overview_of_Cellphone_Data_Cleaning_Functions.pdf)

The following files performed are the subnational_pop_ests and sa2_2023 files.

Read Phase (000_read_files_BD.R)

We have a code script that contains all the imported libraries needed for the pipeline. Once we have that we have a read files where it reads and writes out the whole two files. I created a function that I can easy use to call. Both files get read and it also checks whether or not the files does exist in the directory. Then writes the files and creates a dataframe for the overall data file. While not at the best of it, we then use the cleaning to proceed.

Clean Phase (002_clean_files_BD.R)

The cleaning phase was to check any data quality issues that was in the two files. There was names that was invalid and not correct to define the columns, missing values that have NA's, duplicate data which were repetitive. The first process was to call the read files so that it knows what data file it needs to read and clean.

1. sa2_2023.csv Clean

The sa2_2023 had names that doesn't define the columns. The names were "Clasification report" and "...2". There rows that didn't need to be part of the final dataset such as the first 6. So we had to rename them to fit the columns. The names are "Code" which represents the code number corresponding to the region name. "Name" represents the name of the region. Then deleted the first 6 rows as they were not relevant to the dataset.

2. subnational_pop_ests.csv Clean

The subnatinal_pop_ests had two duplicates that had the same meaning, but the duplicated column had NA all the way. Had to check if every Area to which is similar to sa2_code had a number and no other type of data. The columns needed to be deleted as they can go, so "Area" and "Observation value" were removed. The Year and Age variables had duplicates but decided to keep it for safety. Some of the variables were kept in case it needed to be used such as the Action and Age. For consistency, the name AREA_POPES_SUB_006 was renamed to "SA2_Code" to match the sa2_2023 data file. The type of data for SA2_Code was changed to integer since it needed to to match the other data files as they were converted to integers. Then for removing any non-integers were done to avoid errors.

The following files performed is urban_rural_to_sa2_concord_2023.

Read Phase (000_initialise_AB.R)

This pipeline begins with loading two key libraries: readr and tidyverse to perform data reading and writing operations. Then, it captures the current system date to create a unique, time- stamped filename when saving the processed data. Then it reads the input CSV file to keep it in memory in case it is needed for further processing or analysis and written to a new file which omits row names to improve compatibility with other files. However, since most of the information was not required, further analysis of the file was halted.

Read_phase(000_read_CONCORD_AB.R)

This pipeline is read a CSV file “urban_rural_to_sa2_concord_2023.csv”, along with a boolean to save the resulting data as a RDS file. The input and output file paths are constructed dynamically with the specified folder and filenames, which includes the date, ensuring that the output file is unique to the day it is created, helping with version control and avoids files not to be overwritten. It further checks that no operations are performed on missing files and removes existing RDS file of the same name, ensuring that outdated or incorrect files are not in the directory. Eventually, it extracts the column specification, which allows the user to inspect or work with the column types later during processing. The resulting dataset is saved as an RDS file, which is efficient for storing R data objects and often preferred during processing.

The following files performed are the sa2_ta_concord_2023 and urban_rural_to_indicator_2023 files.

Read_phase(000_initialise_MM.R)

The pipeline begins by conditionally loading packages only if it is required, which makes the script portable and reproducible on different systems. This pipeline processes 2 specific CSV files which are processed separately.

The first input file sa2_ta_concord_2023.csv and output file is dynamically named to ensure version control. Following which it checks if the file exists or halts if the file is missing. To prevent overwriting, it removed the RDS file with the same name. The concordance data is read into a tibble and the RDS file is compressed for optimized for loading in following workflows. The final processed data returns a tibble for immediate use.

The second input file urban_rural_to_indicator_2023.csv has defined output and RDS files like before along with including the current date for clear version control. The pipeline checks in case of missing files to avoid further errors. This also removes pre – existing RDS files with the same name to avoid confusion from older versions. Following, this is read into memory as a tibble in order to be easily manipulated further and saved as an RDS file for long term storage and returns for downstream analysis.

Clean_phase (002_cleam_MM.R)

The pipeline below is designed to save the cleaned data to new CSV files.

The first input file, sa2_ta_concord_2023.csv is dynamically generated to ensure flexibility. We define new columns to provide clear and meaningful identifiers for each column, thus improving readability and maintaining the data. Further, we skipped the first 7 rows to apply the new column names and saved to the new CSV file clean_sa2_ta_concord_data.csv.

The second input file urban_rural_to_indicator_2023.csv is dynamically generated to generate flexibility while handling further analysis. We define new column names for each data field, making sure that the dataset is easily interpretable, and the information is well – organised. Again, the first seven rows are skipped, and the new column names are applied, removing an unnecessary column to keep only relevant columns for analysis. Finally, the cleaned dataset is saved to the new CSV file clean_urban_rr_indicator_data.csv

Building of the Map

To identify the CBD regions of Auckland, Wellington and Christchurch, the process was completed in three steps:

1. Creating a polygon of the CBD:

To get a clear picture of which regions exactly are covered to give the polygon of each CBD, we have found the boundary region coordinates of each CBD, which gave CBD polygon on the map of New Zealand.

2. Finding the SA2 Codes for the CBD:

The polygon found in the previous step created a boundary that could be used to find the corresponding SA2 codes. To help us graph the SA2 codes, we used a map of New Zealand comprised of polygons that surround the Statistical Areas (SA2). This map was found on StatNZ, and by clicking on the polygons that are within the polygon found earlier, we found the SA2 codes.

3. Creating a dataframe for the CBD:

After finding the correct SA2 codes for each CBD, we create a dataframe for Auckland, Wellington and Christchurch CBD by filtering out data that wasn't within the CBD boundaries.

References

1. Ewing, D. (2024). Narrative and debugging overview of cellphone data cleaning functions.